

A
Project Report on
STUDENT REMEDIAL DASHBOARD
Innovation & Entrepreneurship Hub for Educated Rural (SURE
Trust– IERY)



Under the Esteemed Guidance of
Ms. Akshara mam
(Software Developer at propel Technology)

Submitted By
N. SURYA CHARAN

Project Duration March 2026– April 2026

Table of Contents

CH.NO.	INDEX	PAGE NO.
	ABSTRACT	6
	LIST OF FIGURES	7
1	INTRODUCTION	8
	1.1 Background	8
	1.2 Motivation	8
	1.3 Problem Statement	8
	1.4 Objectives	9
	1.5 Scope of the project	9
	1.6 Report organization	9
2	Existing System	10
	2.1 Existing System	10
	2.2 Proposed System	11-12
3	REQUIREMENT ANALYSIS	13
	3.1 System Requirement	13
	3.2 Software Specifications	14-15
4	METHODOLOGY	16
	4.1 Methodology Steps and Explanations	16-17
	4.2 Source Code	17-20
	4.3 Results	21-28
5	CONCLUSIONS AND REFERENCES	
	5.1 Conclusions	29
	5.2 References	30

ABSTRACT

The project titled “STUDENT REMEDIAL DASHBOARD” is a web-based application developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js) to address the lack of transparency in academic result evaluation systems. In the existing system, students can only view external examination marks, while internal assessment components such as assignments, mid exams, and attendance are not visible.

The proposed system provides a centralized platform where students can securely login and access their complete academic performance, including internal marks, external marks, total scores, and pass or remedial status. The system also clearly displays the reason for remedial results, ensuring full transparency.

The backend is built using Node.js and Express.js, handling APIs, business logic, and data processing, while MongoDB is used for storing student records and marks. The frontend is developed using React.js, providing an interactive and user-friendly dashboard.

This system reduces confusion, improves student awareness, and enhances decision-making by presenting clear and structured academic results. It demonstrates how modern web technologies can be used to build scalable and efficient educational solutions.

This system eliminates ambiguity in result evaluation, reduces student confusion, and promotes better academic awareness. By providing detailed insights into performance, it enables students to identify their weak areas and take necessary improvements. Furthermore, the use of modern web technologies ensures scalability, security, and ease of maintenance.

In conclusion, the STUDENT REMEDIAL DASHBOARD serves as an effective solution to enhance transparency in academic result management. It bridges the gap between students and evaluation processes by delivering clear, accurate, and accessible information. The system can be further extended in the future with features such as performance analytics, mobile application support, and real-time notifications, making it a comprehensive tool for academic institutions.

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 4.3.1	Student Login	21
Figure 4.3.2	Faculty Login	22
Figure 4.3.3	Admin Login	23
Figure 4.3.4	Student Dashboard	24
Figure 4.3.5	Student Print Marksheet	25
Figure 4.3.6	Faculty Dashboard	26
Figure 4.3.7	Faculty Uploading Marks	27
Figure 4.3.8	Admin Publishing Marks	28

CHAPTER 1 INTRODUCTION

1.1 BACKGROUND

In today's academic environment, monitoring student performance and identifying learning gaps is essential for improving educational outcomes. Colleges and universities evaluate students through a combination of internal assessments, such as assignments, mid-semester examinations, attendance, and continuous evaluation, along with external end-semester examinations. However, in many institutions, the current result system lacks transparency and does not provide a clear breakdown of these components.

With the advancement of modern web technologies, there is an opportunity to build systems that provide better visibility and analysis of academic performance. A dashboard-based approach can present data in a structured and user-friendly manner, allowing students to easily understand their results and take corrective actions.

The proposed project, "Student Remedial Dashboard," is designed to address these challenges by providing a centralized web-based platform developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js). The system allows students to log in and view detailed subject-wise performance, including internal marks, external marks, total scores, and remedial status. It also highlights the specific reasons for remedial assignments, enabling students to clearly understand their academic standing.

1.2 MOTIVATION

The motivation behind this project arises from the lack of transparency in the current academic result evaluation system. In many institutions, students are only able to view their final examination marks, while internal assessment components such as assignments, mid-semester exams, attendance, and continuous evaluation remain hidden. This incomplete information makes it difficult for students to understand their overall academic performance.

1.3 PROBLEM STATEMENT

In the current academic evaluation system fail to provide a detailed and structured view of student performance, and there is no centralized platform that clearly explains the reasons behind remedial status. Therefore, there is a need for a transparent and user-friendly system that presents internal marks, external marks, total scores, and explicit reasons for remedial outcomes, enabling students to better understand and improve their academic performance.

1.4 OBJECTIVES

The primary objectives of this project are as follows:

- To develop a web-based Student Remedial Dashboard using the MERN stack (MongoDB, Express.js, React.js, and Node.js).

- To provide secure login functionality for students, faculty, and administrators to access the system.
- To enable faculty to upload and manage internal and external marks of students efficiently.
- To display subject-wise internal marks, external marks, and total scores in a clear and structured manner.
- To automatically calculate pass or remedial status based on predefined academic rules.

1.5 SCOPE OF THE PROJECT

The project focuses on developing a web-based Student Remedial Dashboard that provides transparency in academic result evaluation. The system is designed primarily for educational institutions to help students understand their academic performance through a clear and structured presentation of marks.

The application allows students to view subject-wise internal marks, external examination marks, total scores, and remedial status through a secure login system. Faculty members can upload and manage student marks, while administrators can oversee and maintain the overall system.

1.6 REPORT ORGANIZATION

This document is organized into several chapters. Chapter 2 reviews existing systems and their limitations. Chapter 3 introduces the proposed system and its architecture. Chapter 4 describes the development methodology. Chapter 5 provides a detailed implementation overview. Chapter 6 presents test results and observations. Chapter 7 discusses future enhancements. The final section contains the bibliography.

CHAPTER 2

2.1 EXISTING SYSTEM

2.1.1 Current Result Evaluation System

In most educational institutions, the existing result evaluation system primarily displays only the final examination marks of students. Internal assessment components such as assignments, mid-semester exams, attendance, and continuous evaluation are either not displayed or are communicated separately through faculty members.

Students typically access their results through college portals or notice boards, where only the overall marks or grades are shown. This limited information does not provide an understanding of how the final result was calculated. As a result, students are unable to analyze their academic performance effectively.

The situation becomes more problematic when students are assigned remedial examinations. In such cases, the system does not clearly indicate the reason for failure. Students are left uncertain about whether the issue is due to low internal marks, poor performance in external exams, or both.

2.1.2 Limitations of Existing Systems

Despite being widely used, the current system has several limitations:

- **Lack of Transparency:** Internal marks and evaluation criteria are not clearly visible to students.
- **No Detailed Performance Breakdown:** Students cannot view subject-wise internal and external marks separately.
- **Unclear Remedial Reasons:** The system does not specify why a student has been assigned a remedial exam.
- **Dependency on Faculty:** Students must rely on faculty members to understand their results.
- **Time-Consuming Process:** Clarifying doubts about marks requires manual interaction with staff.
- **No Centralized Dashboard:** There is no single platform that provides complete academic performance in one place.

These limitations highlight the need for a more transparent, structured, and student-friendly system to display and analyze academic results effectively.

2.2 PROPOSED SYSTEM

2.2.1 SYSTEM OVERVIEW

The proposed system, titled “Student Remedial Dashboard,” is a web-based application designed to improve transparency in academic result evaluation. The system provides a centralized platform where students can securely log in and access their complete academic performance in a clear and structured manner.

Unlike the existing system, this application displays detailed subject-wise information, including internal assessment marks, external examination marks, total scores, and overall status (pass or remedial). One of the key features of the system is its ability to clearly indicate the reason for remedial results, such as insufficient internal marks, low external exam performance, or both.

2.2.2 Key Components

2.2.2.1 Web Interface (Frontend)

The user interacts with the system through a responsive and user-friendly web interface developed using React.js. The dashboard provides an intuitive layout for students, faculty, and administrators. It includes pages for:

- User registration and login
- Viewing subject-wise marks (internal and external)
- Displaying total scores and remedial status
- Dashboard visualization for easy understanding of performance

2.2.2.2 User Authentication (MongoDB Database)

To ensure secure access, the system includes an authentication module. User credentials such as username and password are securely stored in MongoDB. Passwords are encrypted using hashing techniques for security.

2.2.2.3 Marks Management Module

Faculty members can upload and update internal and external marks of students through the system. The backend, developed using Node.js and Express.js, processes this data and stores it in MongoDB. The system automatically calculates total marks based on predefined rules and determines whether a student has passed or requires a remedial examination.

2.2.2.4 Backend Logic (Node.js and Express.js)

The backend of the system is developed using Node.js and Express.js. It handles all core functionalities including API requests, data processing, result calculation, and database operations. The backend communicates with the MongoDB database to store and retrieve student information, marks, and result details.

It is responsible for calculating total marks based on internal and external components and determining pass or remedial status according to predefined academic rules. The backend also ensures secure data handling, role-based access control, and smooth communication between the frontend and database.

2.2.3 System Architecture

The system follows a client-server architecture based on the MERN stack with the following workflow:

- The client (React frontend) sends requests through the browser.
- The frontend communicates with the backend using RESTful APIs (Axios/HTTP requests).

- The Node.js and Express server processes the request and performs necessary operations.
- The server interacts with the MongoDB database to fetch or store student data and marks.
- The backend calculates total marks and determines remedial status.
- The processed data is sent back to the frontend.
- The frontend displays the results in a structured dashboard format to the user.

This architecture ensures scalability, efficiency, and smooth data flow between components.

2.2.4 Advantages of the Proposed System

- Provides complete transparency by displaying internal and external marks separately
- Clearly indicates the reason for remedial examinations
- Reduces confusion and dependency on faculty for result clarification
- Enables students to identify weak areas and improve performance
- Offers a centralized and user-friendly dashboard for result analysis
- Scalable system that can support multiple users and large datasets
- Can be extended with features like analytics, notifications, and mobile applications

The proposed system significantly improves the clarity and efficiency of academic result evaluation compared to existing methods.

CHAPTER 3

REQUIREMENT ANALYSIS

The development of the “Student Remedial Dashboard” follows a modular and structured approach using the MERN stack. The system is divided into key modules such as user authentication, marks management, result processing, and dashboard visualization. Each module is developed and tested independently before being integrated into a complete web application.

3.1 System Requirement

3.1.1 Hardware Specifications

- CPU: Minimum Intel i3 processor or equivalent (Intel i5 or above recommended)
- RAM: Minimum 4 GB (8 GB recommended for better performance)
- Storage: 500 GB SSD (for quick data access and document storage)
- Network: Stable internet connection for accessing the web application

3.1.2 Software Requirements

Frontend

- Operating System: Windows/Linux/macOS
- Web Browser: Google Chrome, Mozilla Firefox, Microsoft Edge
- Technologies Used: React.js, HTML, CSS, JavaScript

BACKEND

- Operating System: Ubuntu/Linux/Windows
- Runtime Environment: Node.js
- Framework: Express.js
- Database: MongoDB
- Database Tool: MongoDB Compass
- API Communication: REST APIs (HTTP/JSON)

3.1.3 Software Libraries and Frameworks Required

- Express.js – for building the backend server and handling routing
- Mongoose – for interacting with MongoDB and managing schemas
- CORS – for enabling cross-origin requests between frontend and backend
- Body-parser / Express.json – for handling request data in JSON format
- bcrypt – for password hashing and secure authentication
- jsonwebtoken (JWT) – for implementing secure user authentication and authorization

Frontend

- React.js – for building the user interface
- React Router – for navigation between pages
- Axios – for making API requests to the backend

3.2 Software Specifications

3.2.1 MongoDB (Database)

Purpose: MongoDB is used to store student information, including user credentials.

Access Layer: Managed using Mongoose.

Features: Efficient storage and retrieval of large amounts of data.

3.2.2 Node and Express.js (Backend Framework)

Purpose: The backend is responsible of handling user authentication student records.

Features: Supports middleware for request handling and security.

Advantages: Fast and efficient due to non-blocking architecture.

3.2.3 HTML + REACT (Frontend Template Engine)

Purpose: Renders dynamic web pages such as login, transcript viewer, and chatbot interface.

Features: Easy syntax, integrates directly with Flask backend.

Supports: Form submission, chat message rendering, and output display.

3.2.4 Node.js (Runtime Environment)

Purpose: Node.js serves as the runtime environment for backend JavaScript code.

Features: Efficient handling of multiple client requests.

Compatibility: Cross-platform, open-source.

CHAPTER 4 METHODOLOGY

The methodology adopted for this project follows a modular and iterative development approach using the MERN stack. The project is divided into key components such as user authentication, marks management, result processing, and dashboard visualization. Each module is designed, implemented, and tested independently before being integrated into a complete system.

4.1 Methodology Steps and Explanations

- Requirement Analysis & Technology Selection
- Environment Setup
- User Authentication Module
- Marks Management Module
- Result Processing Module
- Dashboard and Frontend Development
- API Integration (Frontend–Backend Communication)
- Error Handling and Logging
- Testing and Iteration

4.1.1 Requirement Analysis & Technology Selection

- Identified the problem: lack of transparency in academic result evaluation and unclear reasons for remedial status.
- Chose relevant technologies:
- MongoDB: for storing student data and marks.
- Express.js: for backend routing and API handling.
- React.js: for building a dynamic user interface.
- Node.js: as the runtime environment for backend development.
- Defined functional and non-functional requirements.

4.1.2 Environment Setup

- Installed Node.js and npm for backend development.
- Set up React.js application using create-react-app.
- Installed required dependencies using npm (Express, Mongoose, Axios, bcrypt, JWT, etc.)
- Configured MongoDB database and connected it using Mongoose.
- Created project structure for frontend and backend modules.

4.1.3 User Authentication Module

- Developed user registration and login pages using React.js forms.
- Implemented backend APIs using Express.js for authentication.

- Stored user credentials securely in MongoDB.
- Used bcrypt for password hashing and security.

4.1.4 Marks Management Module

- Created interfaces for faculty to upload and update student marks.
- Stored internal and external marks in MongoDB.
- Implemented validation to ensure correct data entry.

4.1.5 Result Processing Module

- Designed logic to calculate total marks based on internal and external scores.
- Implemented conditions to determine pass or remedial status.
- Identified and highlighted reasons for remedial status (low internal marks, low external marks, or both).
- Stored processed results in the database for display.

4.1.6 Dashboard and Frontend Development

- Developed a responsive dashboard using React.js.
- Designed user-friendly interface for easy navigation and understanding.
- Integrated frontend with backend APIs using Axios for real-time data fetching.

4.2 SOURCE CODE

User Registration API

```
const express = require('express'); const router = express.Router(); const User =
require('./models/User'); const bcrypt = require('bcryptjs'); const jwt =
require('jsonwebtoken');
router.post('/register', async (req, res) => {  const { username, password } =
req.body;  if (!username || !password) {    return res.status(400).json({
message: "All fields are required" });  }  const existingUser = await
User.findOne({ username });  if (existingUser) {    return res.status(400).json({
message: "User already exists" });  }  const hashedPassword = await
bcrypt.hash(password, 10);  const newUser = new User({ username, password:
hashedPassword });  await newUser.save();  res.status(201).json({ message: "User
registered successfully" }); });
```

User Login API

```
router.post('/login', async (req, res) => {  const { username, password } =
req.body;  const user = await User.findOne({ username });  if (!user) return
res.status(400).json({ message: "Invalid credentials" });  const isMatch = await
bcrypt.compare(password, user.password);  if (!isMatch) return
res.status(400).json({ message: "Invalid credentials" });  const token =
jwt.sign({ id: user._id }, "SECRET_KEY");  res.json({ token, message: "Login
successful" }); });
```

User Schema

```
const mongoose = require('mongoose'); const userSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true }, password: { type:
  String, required: true } }); module.exports = mongoose.model("User", userSchema);
```

Result Calculation Logic

```
function calculateResult(internal, external) { const total = internal + external;
let status = "Pass"; let reason = ""; if (internal < 20) { status =
"Remedial"; reason = "Low Internal Marks"; } else if (external < 40) {
status = "Remedial"; reason = "Low External Marks"; } return { total,
status, reason }; }
```

Dashboard Component

```
import React, { useEffect, useState } from 'react'; import axios from 'axios';
const Dashboard = () => { const [data, setData] = useState([]); useEffect(() =>
{ axios.get("http://localhost:5000/api/results") .then(res =>
setData(res.data)) .catch(err => console.log(err)); }, []); return (
<div> Student Dashboard </div> ); }; export default Dashboard;
```

Axios API Call

```
import axios from 'axios'; const fetchResults = async () => { try { const
response = await axios.get("http://localhost:5000/api/results");
console.log(response.data); } catch (error) { console.error("Error fetching",
error); } };
```

4.3 RESULT

4.3.1 Student Login

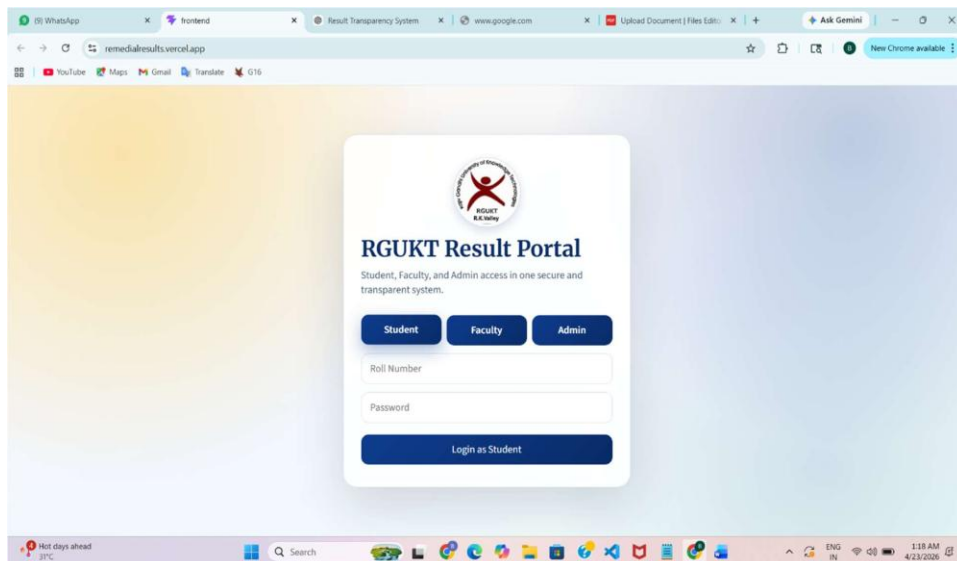
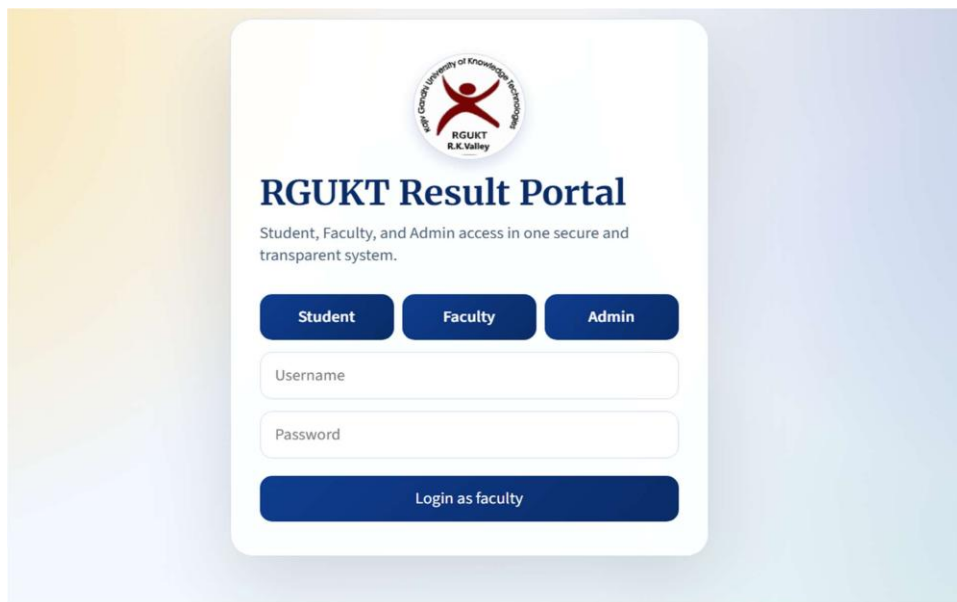


Figure 4.3.1: Student Login

The Student Login module provides secure access to the Student Remedial Dashboard. It allows students to log in using their registered username and password to view their academic performance. The authentication process is handled through backend APIs developed using Node.js and Express.js, while user credentials are securely stored in the MongoDB database.

4.3.2 Faculty Login



The image shows a web interface for the RGUKT Result Portal. At the top center is the RGUKT logo, which features a stylized red figure with arms raised, surrounded by the text "RGUKT R.K. Valley" and "Rajasthan Open University of Knowledge Technology". Below the logo, the title "RGUKT Result Portal" is displayed in a bold, dark blue font. Underneath the title, a subtitle reads "Student, Faculty, and Admin access in one secure and transparent system." Below this, there are three dark blue buttons with white text: "Student", "Faculty", and "Admin". The "Faculty" button is highlighted. Below these buttons are two white input fields with light blue borders, labeled "Username" and "Password". At the bottom of the form is a large dark blue button with white text that says "Login as faculty". The entire form is set against a light blue and yellow gradient background.

Figure 4.3.2: Faculty Login

The Faculty Login module provides secure access for faculty members to manage student academic records. After successful login, faculty can upload, update, and maintain internal and external marks of students. The system ensures that only authorized faculty members can modify data, maintaining accuracy and security. This module simplifies the process of result management and supports efficient handling of student performance data.

4.3.3 Admin Login

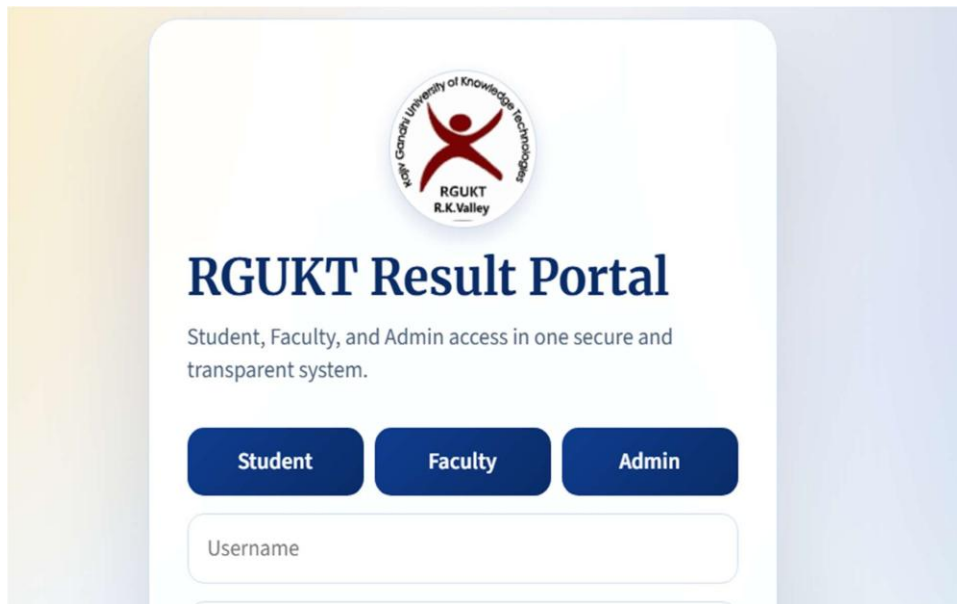


Figure 4.3.3: Admin Login

The Admin Login module provides secure access to the system for administrators. After logging in, the admin can manage overall system operations such as maintaining student records, managing faculty accounts, and overseeing result data. The admin has full control over the system to ensure data accuracy, consistency, and proper functioning. This module plays a key role in maintaining the integrity and smooth operation of the Student Remedial Dashboard.

4.3.4 Student Dashboard

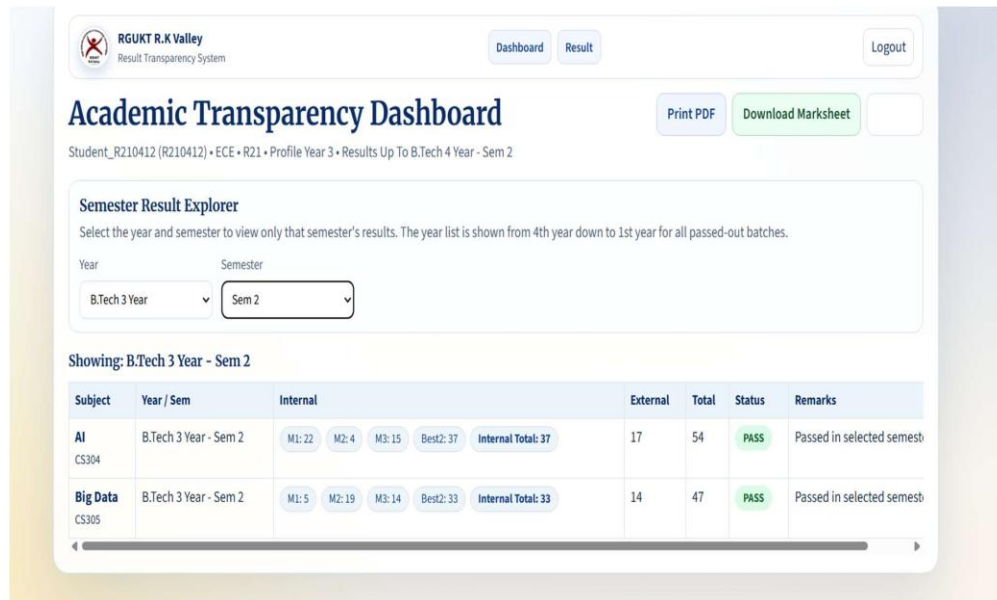


Figure 4.3.4: Student Dashboard

The Student Dashboard provides a clear and structured view of the student's academic performance. After logging in, students can view subject-wise internal marks, external examination marks, total scores, and overall status (pass or remedial). The dashboard is designed to present information in a simple and easy-to-understand format.

One of the key features of this dashboard is that it clearly highlights the reason for remedial status, such as low internal marks, poor external performance, or both. This helps students identify their weak areas and take necessary steps for improvement.

4.3.5 Student Printing Marksheet

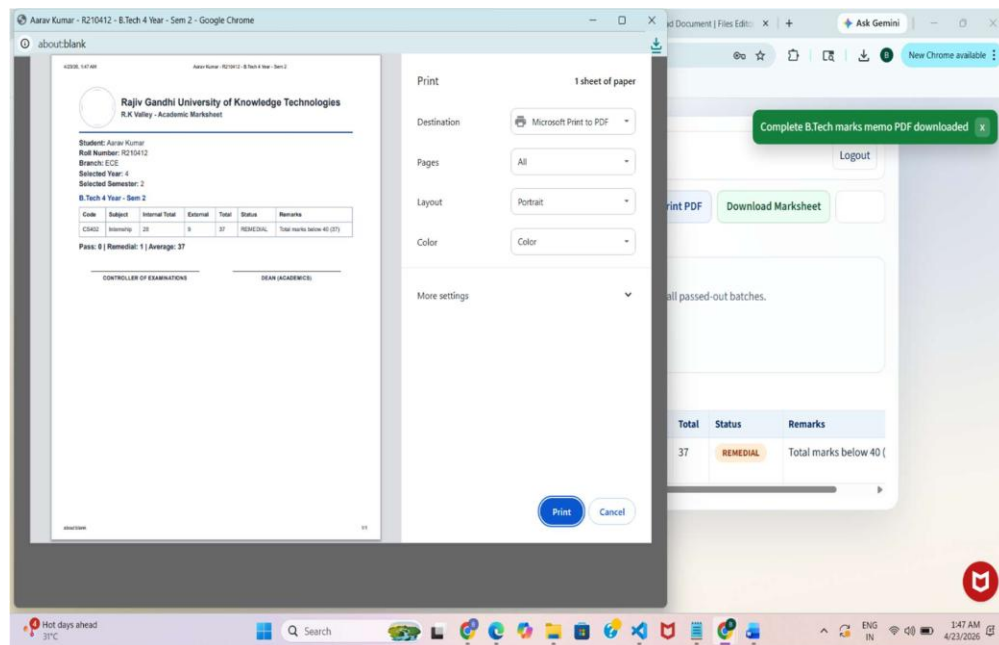


Figure 4.3.5: Student Printing Marksheet

The Student Remedial Dashboard provides a feature that allows students to download or print their marksheet for future reference. By clicking the “Print” or “Download Marksheet” option, the system generates a structured report containing subject-wise internal marks, external marks, total scores, and overall result status.

The marksheet is presented in a clear and organized format, making it easy to understand and suitable for official or personal use. This feature helps students maintain a record of their academic performance without the need to manually collect or request documents from the institution.

4.3.6 Faculty Dashboard

RGUKT R.K Valley
Result Transparency System

Dashboard Marks Entry Logout

Result Publication and Internal Marks Control

Logged in as faculty (faculty) Logout

Year Semester Branch Subject

2nd Year Semester 2 CSE CS205 - OOPS Download Class Marksheet

Loaded 360 students for Year 2 Semester 2 (CSE)

Faculty Marks Entry Table

Enter Mid1, Mid2, Mid3 (each out of 20) and Sem Marks (out of 60). Best 2 mids are considered for internal total out of 40.

Roll Number	Name	Mid 1	Mid 2	Mid 3	Sem Marks	Assignment	Attendance	Continuous Eval	Action
R220001	Student_R220001								<button>Save</button>
R220002	Student_R220002								<button>Save</button>
R220003	Student_R220003								<button>Save</button>
R220004	Student_R220004								<button>Save</button>
R220005	Student_R220005								<button>Save</button>

Figure 4.3.6: Faculty Dashboard

The Faculty Dashboard provides an interface for faculty members to manage student academic records efficiently. After logging in, faculty can add, update, and maintain internal and external marks for students based on their respective subjects.

The system allows faculty to enter marks in a structured format, ensuring accuracy and consistency in data entry. Once the marks are submitted, the backend automatically processes the data to calculate total scores and determine the student's pass or remedial status.

This dashboard simplifies the evaluation process, reduces manual effort, and ensures that student performance data is updated in real-time. It also plays a crucial role in maintaining transparency by enabling accurate and timely input of marks into the system.

4.3.7 Faculty Uploading Marks file

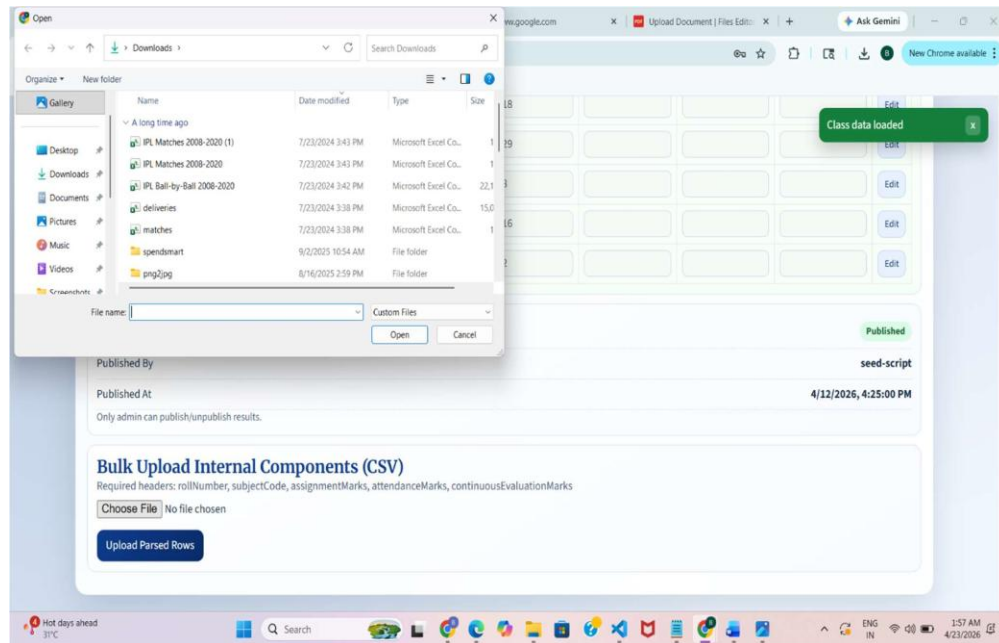


Figure 4.3.7: Faculty Uploading Marks

The system provides a convenient feature that allows faculty members to upload student marks in bulk using an Excel (.xlsx) file. This functionality simplifies the process of entering marks for multiple students by eliminating the need for manual data entry.

Faculty can download a predefined Excel template, fill in the required details such as student ID, subject, internal marks, and external marks, and upload the file through the dashboard. The system validates the uploaded data to ensure correctness and consistency before storing it in the database.

Once the file is successfully uploaded, the backend automatically processes the marks, calculates total scores, and updates the pass or remedial status of each student. This feature saves time, reduces errors, and ensures efficient management of large volumes of student data.

4.3.8 Admin Publishing Marks

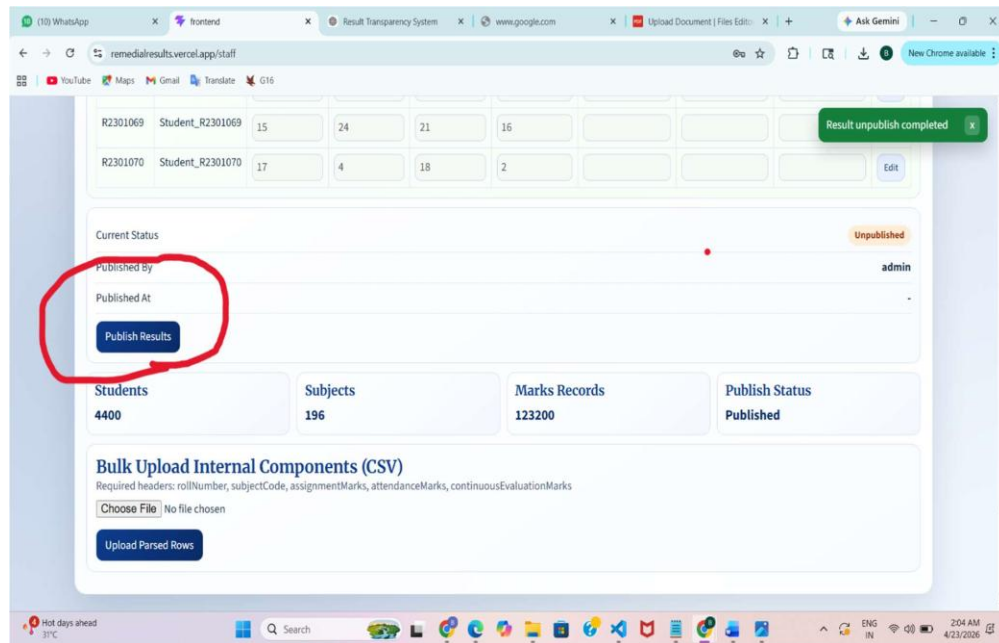


Figure 4.3.8: Admin Publishing Marks

The Admin Dashboard provides complete control over the Student Remedial Dashboard system. The administrator has full access to manage all system operations, including student records, faculty accounts, subject details, and academic data.

One of the key responsibilities of the admin is to review and publish student results. After faculty members upload marks, the admin verifies the data and officially publishes the results, making them visible to students through their dashboards. This ensures accuracy and prevents unauthorized or incomplete data from being displayed.

The admin can also update or modify records when necessary, ensuring data consistency and reliability. With centralized control, this module maintains the integrity, security, and smooth functioning of the entire system.

CHAPTER 5

CONCLUSION AND REFERENCES

5.1 CONCLUSIONS

The project “Student Remedial Dashboard” successfully achieves its objective of improving transparency in the academic result evaluation system. By providing a clear and structured view of internal marks, external marks, total scores, and remedial status, the system helps students better understand their academic performance.

One of the key contributions of this project is the ability to clearly indicate the reason for remedial results, whether due to low internal marks, poor external performance, or both. This eliminates confusion and enables students to identify their weak areas and take necessary steps for improvement.

The project demonstrates the effective use of modern web technologies such as the MERN stack (MongoDB, Express.js, React.js, and Node.js) to build a scalable, efficient, and user-friendly application. The system also provides role-based access for students, faculty, and administrators, ensuring secure and organized data management.

The modular design of the system allows for easy maintenance and future enhancements, such as performance analytics, graphical reports, mobile application support, and real-time notifications.

Overall, the Student Remedial Dashboard enhances clarity, reduces dependency on faculty for result explanations, and promotes better academic awareness among students. It serves as a practical solution for modernizing the result management system and contributes to improving the overall educational experience.

5.2 REFERENCES

1. MongoDB Documentation – Official Website <https://www.mongodb.com/docs/>
2. Node.js – Official Documentation <https://nodejs.org/en/docs/>
3. Express.js Framework – Official Documentation <https://expressjs.com/>
4. React.js – Official Documentation <https://react.dev/>
5. Mongoose ODM – Documentation <https://mongoosejs.com/docs/>
6. JSON Web Tokens (JWT) – Introduction and Guide <https://jwt.io/introduction>
7. REST API Concepts – GeeksforGeeks <https://www.geeksforgeeks.org/rest-api-introduction/>
8. HTML, CSS, and JavaScript – MDN Web Docs <https://developer.mozilla.org/>
9. Axios Library for API Calls – Documentation <https://axios-http.com/docs/intro>
10. MERN Stack Development Guide – GeeksforGeeks <https://www.geeksforgeeks.org/mern-stack/>

11. MongoDB Atlas (Cloud Database) – Documentation <https://www.mongodb.com/atlas/database>

12. Full Stack Web Development (MERN) – Tutorial by freeCodeCamp
<https://www.freecodecamp.org/news/tag/mern-stack/>